

Práctica de Bases de Datos Distribuidas

Caso Banco del Austro: guía paso a paso desde cero

Jeremy Ruperto Gaibor Rodríguez

Abstract

Este documento describe, paso a paso, la implementación de un escenario de **fragmentación horizontal** sobre dos motores PostgreSQL independientes (nodo Cuenca y nodo Quito), orquestados con Docker, y consultados de forma unificada mediante una aplicación Spring Boot que enruta las peticiones según el prefijo del número de cuenta. Se documenta también una prueba de tolerancia a fallos que evidencia el comportamiento del sistema cuando uno de los nodos deja de estar disponible.

Contents

1	Verificación rápida del ambiente	3
2	Crear la estructura de carpetas del proyecto	3
2.1	Crear las carpetas desde <code>cmd.exe</code>	3
2.2	Estructura esperada	3
3	Levantar los motores PostgreSQL con Docker	5
3.1	Crear el archivo <code>docker-compose.yml</code>	5
4	Crear las tablas y datos de prueba	5
4.1	Crear los scripts del nodo Cuenca	6
4.1.1	01_schema.sql de Cuenca	6
4.1.2	02_datos.sql de Cuenca	7
4.2	Crear los scripts del nodo Quito	7
5	Levantar los contenedores por primera vez	9
5.1	Asegurarse de que Docker Desktop está corriendo	9
5.2	Ejecutar <code>docker compose up</code>	9
5.3	Verificar los contenedores en la línea de comandos	10
5.4	Verificar los logs de inicialización	10
5.5	Verificar los contenedores en Docker Desktop (interfaz gráfica)	11
6	Verificar los datos con pgAdmin (opción gráfica)	11
6.1	Configurar pgAdmin 4	11
6.1.1	Abrir pgAdmin y crear dos servidores	11
6.1.2	Navegar hasta las tablas	12
7	Crear el proyecto Spring Boot	12
7.1	Generar el proyecto	12
7.2	Elegir Spring Boot y dependencias	14
7.3	Estructura del proyecto que se genera	14
7.4	Cambiar <code>application.properties</code> por <code>application.yml</code>	14

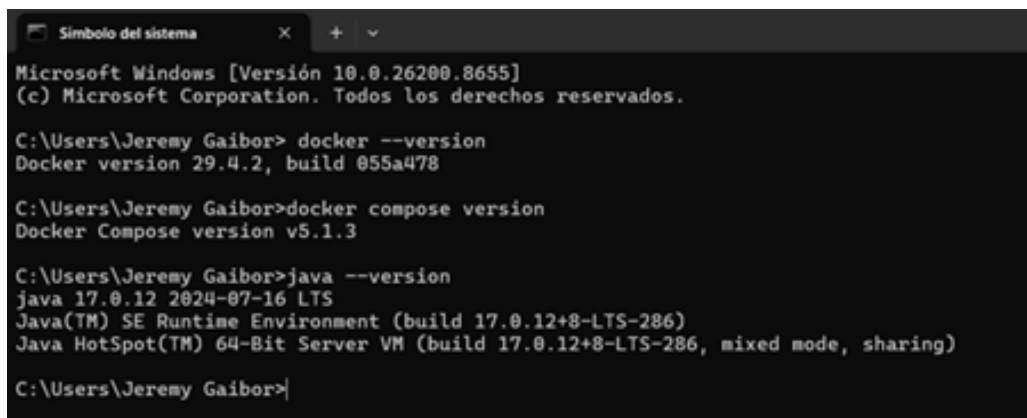
8	Escribir el código Java de la aplicación	16
8.1	Organizar los paquetes	16
8.2	Clase de configuración de los dos <code>DataSource</code>	16
8.2.1	<code>DataSourceConfig.java</code>	16
8.2.2	<code>ConsultaDistribuidaService.java</code>	18
8.2.3	<code>BancoController.java</code>	19
8.3	Estructura del proyecto Java en este punto	20
9	Ejecutar y probar la aplicación	20
9.1	Levantar la aplicación Spring Boot	20
9.2	Probar los endpoints con Postman	20
9.2.1	Primera petición: consultar saldo en el nodo Cuenca	21
9.2.2	Segunda petición: consultar saldo en el nodo Quito	21
9.2.3	Tercera petición: unión de clientes de ambos nodos	22
10	Prueba de tolerancia a fallos	22
10.1	Detener el nodo Quito	22
10.2	Repetir las pruebas con un nodo caído	23
11	Conclusiones	25

1 Verificación rápida del ambiente

Antes de continuar, abra `cmd.exe` y ejecute los siguientes comandos, uno por uno. Cada uno debe imprimir una versión; si alguno responde que `'xxx'` no se reconoce como un comando, entonces el programa correspondiente no está instalado o falta agregar su carpeta al `PATH`.

```
1 docker --version
2 docker compose version
3 java -version
4 mvn -version
```

Listing 1: Comandos de verificación del entorno



```
Simbolo del sistema
Microsoft Windows [Versión 10.0.26200.8655]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Jeremy Gaibor> docker --version
Docker version 29.4.2, build 055a478

C:\Users\Jeremy Gaibor>docker compose version
Docker Compose version v5.1.3

C:\Users\Jeremy Gaibor>java --version
java 17.0.12 2024-07-16 LTS
Java(TM) SE Runtime Environment (build 17.0.12+8-LTS-286)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.12+8-LTS-286, mixed mode, sharing)

C:\Users\Jeremy Gaibor>
```

Figure 1: Verificación del ambiente en `cmd.exe`

2 Crear la estructura de carpetas del proyecto

Antes de tocar Docker o IntelliJ, se prepara la carpeta donde van a vivir todos los archivos de esta práctica: los scripts SQL, el `docker-compose.yml` y luego el proyecto Java.

2.1 Crear las carpetas desde `cmd.exe`

```
1 mkdir C:\uteq
2 mkdir C:\uteq\banco-austro
3 cd C:\uteq\banco-austro
4 mkdir sql-cuenca
5 mkdir sql-quito
```

Listing 2: Creación de la estructura de carpetas

2.2 Estructura esperada

Al finalizar esta sección, la carpeta debe verse así:

```
1 cd C:\uteq\banco-austro
2 dir
```

Listing 3: Verificación de la estructura inicial

```
C:\Users\Jeremy Gaibor>mkdir C:\uteq  
C:\Users\Jeremy Gaibor>mkdir C:\uteq\banco-austro  
C:\Users\Jeremy Gaibor>cd C:\uteq\banco-austro  
C:\uteq\banco-austro>mkdir sql-cuenca  
C:\uteq\banco-austro>mkdir sql-quito  
C:\uteq\banco-austro>|
```

Figure 2: Creación de la raíz y las dos subcarpetas de scripts SQL

```
C:\uteq\banco-austro>cd C:\uteq\banco-austro  
C:\uteq\banco-austro>dir  
El volumen de la unidad C no tiene etiqueta.  
El número de serie del volumen es: C627-7C20  
  
Directorio de C:\uteq\banco-austro  
  
09/07/2026  10:02    <DIR>          .  
09/07/2026  10:02    <DIR>          ..  
09/07/2026  10:02    <DIR>          sql-cuenca  
09/07/2026  10:02    <DIR>          sql-quito  
                0 archivos                0 bytes  
                4 dirs  90.968.883.200 bytes libres  
  
C:\uteq\banco-austro>|
```

Figure 3: Estructura inicial del proyecto

3 Levantar los motores PostgreSQL con Docker

3.1 Crear el archivo docker-compose.yml

Abra VS Code. Vaya al menú **File** → **Open Folder...** y seleccione `C:\uteq\banco-austro`. En el panel izquierdo (*Explorer*) deben aparecer las dos subcarpetas `sql-cuenca` y `sql-quito`. Pulse el botón de nuevo archivo (*New File*), llame al archivo exactamente `docker-compose.yml` y pulse **Enter**.

Copie el siguiente contenido exactamente, respetando los espacios (dos espacios por nivel de indentación) y sin usar tabuladores.

```
1 services:
2
3   db-cuenca:
4     image: postgres:16-alpine
5     container_name: banco-cuenca
6     environment:
7       POSTGRES_DB: banco_cuenca
8       POSTGRES_USER: banco
9       POSTGRES_PASSWORD: banco123
10    ports:
11      - "5432:5432"
12    volumes:
13      - vol-cuenca:/var/lib/postgresql/data
14      - ./sql-cuenca:/docker-entrypoint-initdb.d
15    networks:
16      - bda-net
17
18   db-quito:
19     image: postgres:16-alpine
20     container_name: banco-quito
21     environment:
22       POSTGRES_DB: banco_quito
23       POSTGRES_USER: banco
24       POSTGRES_PASSWORD: banco123
25    ports:
26      - "5433:5432"
27    volumes:
28      - vol-quito:/var/lib/postgresql/data
29      - ./sql-quito:/docker-entrypoint-initdb.d
30    networks:
31      - bda-net
32
33 networks:
34   bda-net:
35     driver: bridge
36
37 volumes:
38   vol-cuenca:
39   vol-quito:
```

Listing 4: `docker-compose.yml` con los dos nodos PostgreSQL

4 Crear las tablas y datos de prueba

Este escenario aplica una **fragmentación horizontal**: cada nodo almacena únicamente las filas cuya oficina coincide con su ubicación. Formalmente, si R es la relación `cuentas`, los fragmentos

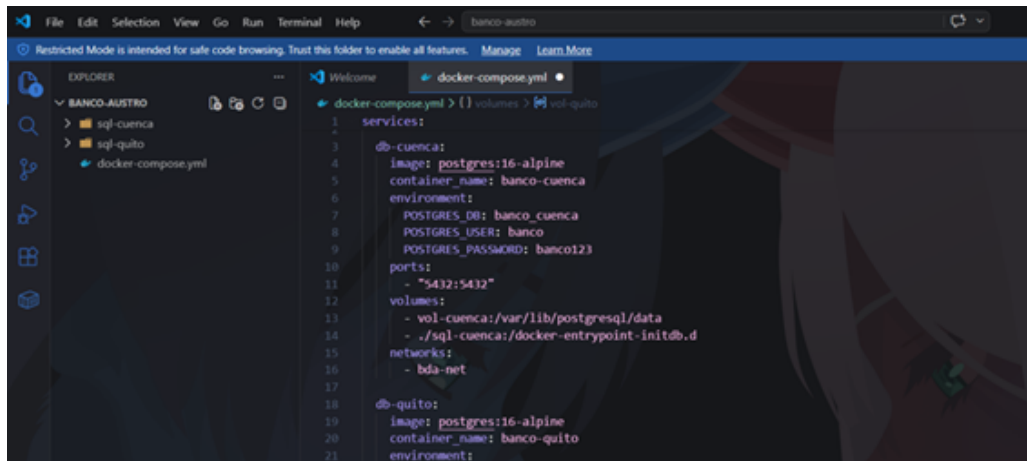


Figure 4: C:\uteq\banco-austro\docker-compose.yml

se definen como

$$R_{\text{Cuenca}} = \sigma_{\text{oficina}='CUENCA'}(R) \quad (1)$$

$$R_{\text{Quito}} = \sigma_{\text{oficina}='QUITO'}(R) \quad (2)$$

de modo que $R = R_{\text{Cuenca}} \cup R_{\text{Quito}}$ y $R_{\text{Cuenca}} \cap R_{\text{Quito}} = \emptyset$.

4.1 Crear los scripts del nodo Cuenca

4.1.1 01_schema.sql de Cuenca

En VS Code, con la carpeta **banco-austro** abierta, haga clic derecho sobre la carpeta **sql-cuenca** y elija *New File*. Nombre el archivo exactamente **01_schema.sql** y presione **Enter**. Copie el siguiente contenido:

```

1  -- Esquema del nodo Cuenca
2  -- Fragmento horizontal: filas con oficina = 'CUENCA'
3
4  CREATE TABLE IF NOT EXISTS clientes (
5      id BIGSERIAL PRIMARY KEY,
6      cedula VARCHAR(10) UNIQUE NOT NULL,
7      nombre VARCHAR(120) NOT NULL,
8      ciudad VARCHAR(60) NOT NULL,
9      creado_en TIMESTAMP NOT NULL DEFAULT NOW()
10 );
11
12 CREATE TABLE IF NOT EXISTS cuentas (
13     id BIGSERIAL PRIMARY KEY,
14     numero VARCHAR(20) UNIQUE NOT NULL,
15     cliente_id BIGINT NOT NULL REFERENCES clientes(id),
16     saldo NUMERIC(15,2) NOT NULL DEFAULT 0.00 CHECK (saldo >= 0),
17     oficina VARCHAR(20) NOT NULL CHECK (oficina = 'CUENCA'),
18     abierta_en TIMESTAMP NOT NULL DEFAULT NOW()
19 );
20
21 CREATE TABLE IF NOT EXISTS transacciones (
22     id BIGSERIAL PRIMARY KEY,
23     cuenta_orig VARCHAR(20) NOT NULL,
24     cuenta_dest VARCHAR(20) NOT NULL,

```

```

25     monto NUMERIC(15,2) NOT NULL CHECK (monto > 0),
26     fecha TIMESTAMP NOT NULL DEFAULT NOW(),
27     estado VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA',
28     oficina VARCHAR(20) NOT NULL CHECK (oficina = 'CUENCA')
29 );
30
31 CREATE INDEX idx_cuentas_cliente ON cuentas(cliente_id);
32 CREATE INDEX idx_transacciones_orig ON transacciones(cuenta_orig);
33 CREATE INDEX idx_transacciones_fecha ON transacciones(fecha);

```

Listing 5: Esquema del nodo Cuenca

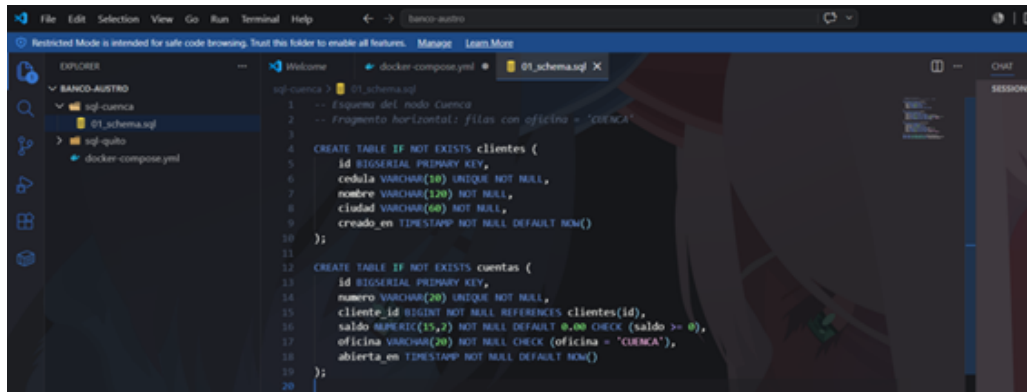


Figure 5: sql-cuenca/01_schema.sql

4.1.2 02_datos.sql de Cuenca

En la misma carpeta `sql-cuenca`, cree un nuevo archivo llamado `02_datos.sql` y pegue lo siguiente:

```

1  -- Datos iniciales del nodo Cuenca
2
3  INSERT INTO clientes (cedula, nombre, ciudad) VALUES
4  ('0101234567', 'Mario Vintimilla', 'Cuenca'),
5  ('0107654321', 'Sofia Vasquez', 'Cuenca'),
6  ('0103456789', 'Andres Cordero', 'Cuenca');
7
8  INSERT INTO cuentas (numero, cliente_id, saldo, oficina) VALUES
9  ('2201000001', 1, 15000.00, 'CUENCA'),
10 ('2201000002', 2, 4200.50, 'CUENCA'),
11 ('2201000003', 3, 850.75, 'CUENCA');
12
13 INSERT INTO transacciones (cuenta_orig, cuenta_dest, monto, oficina)
14     VALUES
15 ('2201000001', '2201000002', 500.00, 'CUENCA'),
16 ('2201000002', '2201000003', 120.25, 'CUENCA');

```

Listing 6: Datos iniciales del nodo Cuenca

4.2 Crear los scripts del nodo Quito

De la misma forma, en la carpeta `sql-quito` cree los archivos `01_schema.sql` y `02_datos.sql`. El esquema es idéntico al de Cuenca, salvo que las restricciones `CHECK` referencian a `'QUITO'`; los datos también son distintos.

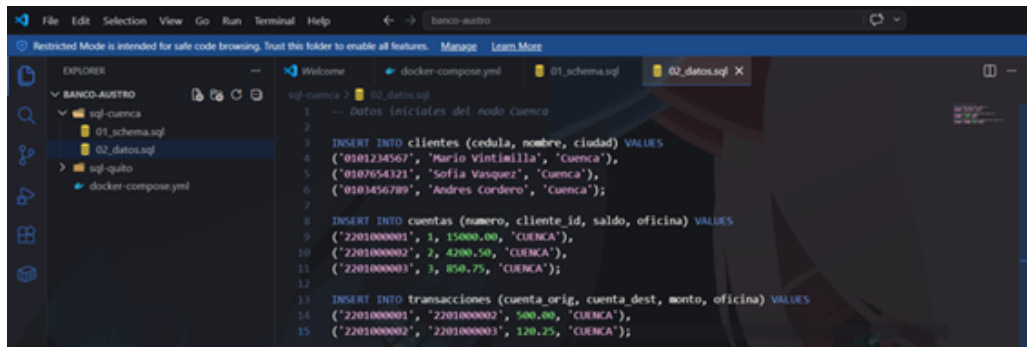


Figure 6: sql-cuenca/02_datos.sql

```

1  -- Esquema del nodo Quito
2  -- Fragmento horizontal: filas con oficina = 'QUITO'
3
4  CREATE TABLE IF NOT EXISTS clientes (
5      id BIGSERIAL PRIMARY KEY,
6      cedula VARCHAR(10) UNIQUE NOT NULL,
7      nombre VARCHAR(120) NOT NULL,
8      ciudad VARCHAR(60) NOT NULL,
9      creado_en TIMESTAMP NOT NULL DEFAULT NOW()
10 );
11
12 CREATE TABLE IF NOT EXISTS cuentas (
13     id BIGSERIAL PRIMARY KEY,
14     numero VARCHAR(20) UNIQUE NOT NULL,
15     cliente_id BIGINT NOT NULL REFERENCES clientes(id),
16     saldo NUMERIC(15,2) NOT NULL DEFAULT 0.00 CHECK (saldo >= 0),
17     oficina VARCHAR(20) NOT NULL CHECK (oficina = 'QUITO'),
18     abierta_en TIMESTAMP NOT NULL DEFAULT NOW()
19 );
20
21 CREATE TABLE IF NOT EXISTS transacciones (
22     id BIGSERIAL PRIMARY KEY,
23     cuenta_orig VARCHAR(20) NOT NULL,
24     cuenta_dest VARCHAR(20) NOT NULL,
25     monto NUMERIC(15,2) NOT NULL CHECK (monto > 0),
26     fecha TIMESTAMP NOT NULL DEFAULT NOW(),
27     estado VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA',
28     oficina VARCHAR(20) NOT NULL CHECK (oficina = 'QUITO')
29 );
30
31 CREATE INDEX idx_cuentas_cliente ON cuentas(cliente_id);
32 CREATE INDEX idx_transacciones_orig ON transacciones(cuenta_orig);
33 CREATE INDEX idx_transacciones_fecha ON transacciones(fecha);

```

Listing 7: Esquema del nodo Quito (01_schema.sql)

```

1  -- Datos iniciales del nodo Quito
2
3  INSERT INTO clientes (cedula, nombre, ciudad) VALUES
4  ('1701234567', 'Pedro Jimenez', 'Quito'),
5  ('1709876543', 'Luisa Paredes', 'Quito'),
6  ('1704567890', 'Carla Espinoza', 'Quito');
7
8  INSERT INTO cuentas (numero, cliente_id, saldo, oficina) VALUES

```



```

9  ('1701000001', 1, 8000.00, 'QUITO'),
10 ('1701000002', 2, 2500.00, 'QUITO'),
11 ('1701000003', 3, 975.40, 'QUITO');
12
13 INSERT INTO transacciones (cuenta_orig, cuenta_dest, monto, oficina)
   VALUES
14 ('1701000001', '1701000002', 300.00, 'QUITO'),
15 ('1701000003', '1701000001', 75.00, 'QUITO');

```

Listing 8: Datos iniciales del nodo Quito (02_datos.sql)

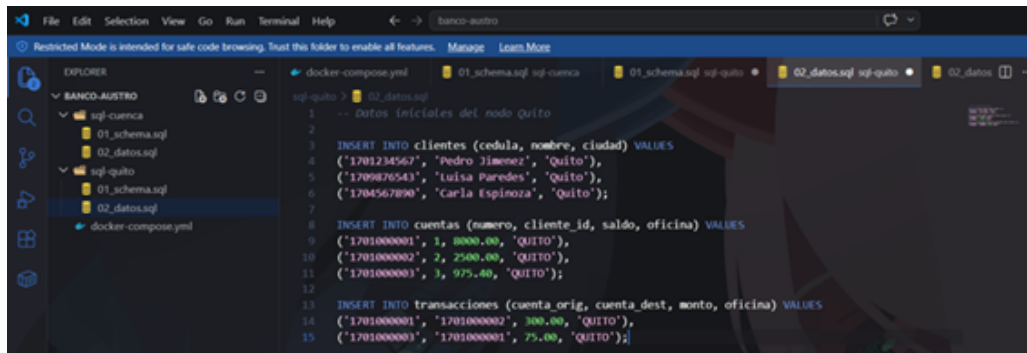


Figure 7: Scripts del nodo Quito

5 Levantar los contenedores por primera vez

5.1 Asegurarse de que Docker Desktop está corriendo

Abra Docker Desktop desde el menú de Inicio. Espere a que el ícono en la barra de tareas quede en verde (o a que la ventana muestre *Docker Desktop is running*). Sin Docker Desktop corriendo, todos los comandos siguientes fallan con el error `error during connect`.

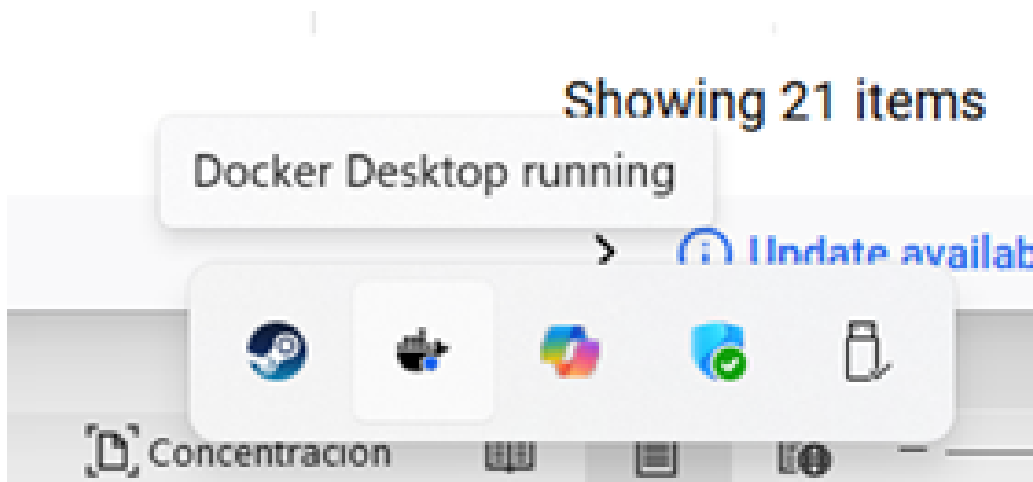


Figure 8: Docker Desktop en ejecución

5.2 Ejecutar docker compose up

En `cmd.exe`, cambie a la carpeta del proyecto y levante los servicios:

```

1 cd C:\uteq\banco-austro
2 docker compose up -d

```

Listing 9: Levantar los contenedores

```

C:\uteq\banco-austro>cd C:\uteq\banco-austro
C:\uteq\banco-austro>docker compose up -d
[+] up 5/5
✓Network banco-austro_bda-net      Created      0.0s
✓Volume banco-austro_vol-cuenca    Created      0.0s
✓Volume banco-austro_vol-quito     Created      0.0s
✓Container banco-quito             Started      0.7s
✓Container banco-cuenca            Started      0.7s

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop:///dashboard/logs

C:\uteq\banco-austro>

```

Figure 9: Salida de docker compose up

La primera vez, Docker descarga la imagen `postgres:16-alpine` (unos 250 MB). Espere un mensaje similar a:

```

1 [+] Running 5/5
2 - Network banco-austro_bda-net      Created
3 - Volume "banco-austro_vol-cuenca"  Created
4 - Volume "banco-austro_vol-quito"   Created
5 - Container banco-cuenca            Started
6 - Container banco-quito             Started

```

Listing 10: Mensaje esperado tras levantar los servicios

5.3 Verificar los contenedores en la línea de comandos

Todavía en `cmd.exe`, ejecute:

```

1 docker compose ps

```

Listing 11: Estado de los contenedores

```

C:\uteq\banco-austro>docker compose ps
NAME                IMAGE              COMMAND              SERVICE    CREATED        STATUS        PORTS
banco-cuenca        postgres:16-alpine "docker-entrypoint.s- db-cuenca    2 minutes ago  Up 2 minutes  0.0.0.0:5432->5432/tcp
banco-quito         postgres:16-alpine "docker-entrypoint.s- db-quito     2 minutes ago  Up 2 minutes  0.0.0.0:5433->5433/tcp
C:\uteq\banco-austro>

```

Figure 10: Contenedores en línea

5.4 Verificar los logs de inicialización

Muestre las últimas líneas del log de cada contenedor:

```

1 docker logs banco-cuenca --tail 30
2 docker logs banco-quito --tail 30

```

Listing 12: Consulta de logs de cada nodo

5.5 Verificar los contenedores en Docker Desktop (interfaz gráfica)

Abra la ventana de Docker Desktop. En el menú lateral izquierdo, seleccione *Containers*. Debe ver un grupo llamado **banco-austro** con dos entradas: **banco-cuenca** y **banco-quito**, ambas con estado *Running* (punto verde).

Haga clic en cualquiera de los dos contenedores y luego en la pestaña *Logs* para inspeccionar visualmente lo mismo que se vio con `docker logs`. La pestaña *Terminal* le da una shell dentro del contenedor si necesita entrar manualmente.

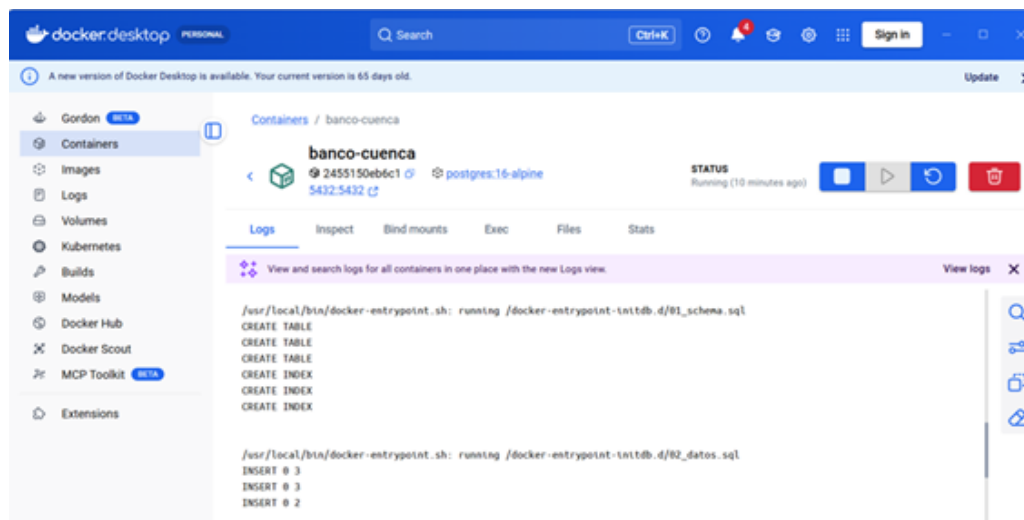


Figure 11: Verificación de los contenedores en Docker Desktop

6 Verificar los datos con pgAdmin (opción gráfica)

Antes de escribir una sola línea de código Java conviene comprobar, con una herramienta gráfica de bases de datos, que los dos motores contienen exactamente el fragmento que deberían. Se muestran dos alternativas equivalentes: pgAdmin 4 (herramienta dedicada) o el panel *Database* de IntelliJ IDEA (integrado en el IDE).

6.1 Configurar pgAdmin 4

6.1.1 Abrir pgAdmin y crear dos servidores

Abra pgAdmin 4. Se abrirá en su navegador (<http://127.0.0.1:PUERTO>). En el panel izquierdo, haga clic derecho sobre *Servers* → *Register* → *Server....* Complete la primera conexión con los siguientes datos y pulse *Save*:

Table 1: Parámetros de conexión del servidor *Banco Austro Cuenca*

Pestaña	Campo / valor
General	Name: Banco Austro Cuenca
	Host name/address: localhost
	Port: 5432
Connection	Maintenance database: banco_cuenca
	Username: banco
	Password: banco123

Repita el mismo proceso para el segundo servidor, con nombre **Banco Austro Quito**, puerto 5433 y base **banco_quito**.

6.1.2 Navegar hasta las tablas

En el panel izquierdo, expanda *Servers* → *Banco Austro Cuenca* → *Databases* → *banco_cuenca* → *Schemas* → *public* → *Tables*. Debe ver tres tablas: **clientes**, **cuentas** y **transacciones**. Repita en el servidor de Quito.

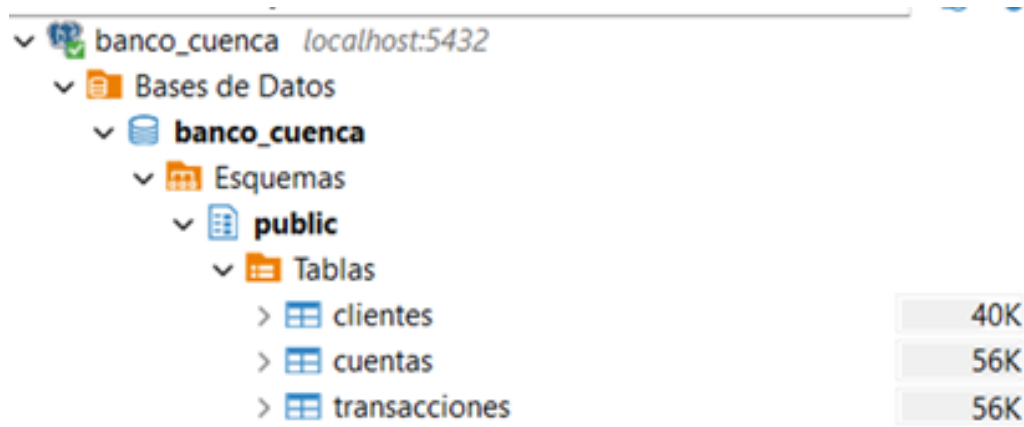


Figure 12: Navegar hasta las tablas del nodo Cuenca

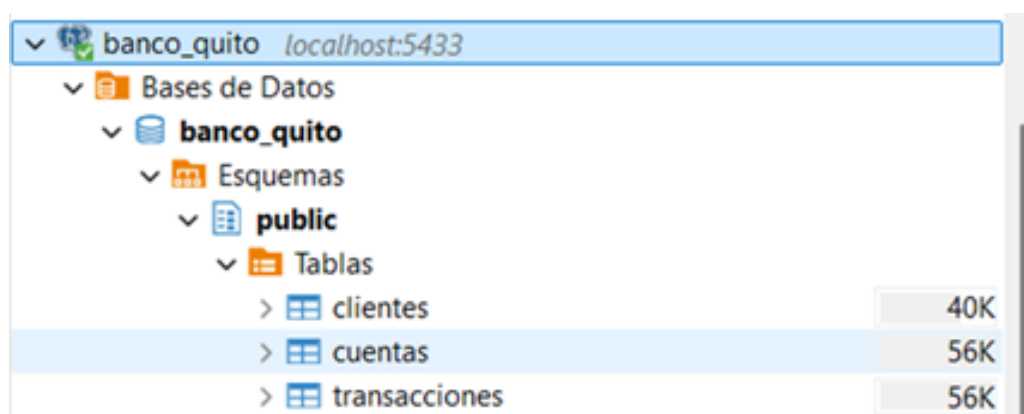


Figure 13: Navegar hasta las tablas del nodo Quito

En pgAdmin, con el servidor de Cuenca seleccionado, abra el editor de consultas: botón *Query Tool* (ícono con un rayo). Escriba y ejecute:

```
1 SELECT c.numero, cl.nombre, c.saldo, c.oficina
2 FROM cuentas c
3 JOIN clientes cl ON cl.id = c.cliente_id
4 ORDER BY c.numero;
```

Listing 13: Consulta de verificación sobre el nodo Cuenca

7 Crear el proyecto Spring Boot

7.1 Generar el proyecto

Abra IntelliJ IDEA. En la pantalla de bienvenida pulse *New Project*. En el panel izquierdo elija *Spring Boot* (o *Spring Initializr*, según la versión). Complete el asistente con:

Table 2: Configuración del proyecto Spring Boot

Campo	Valor
Name	banco-austro
Location	C:\uteq\banco-austro
Language	Java
Type	Maven
Group	ec.edu.uteq
Artifact	banco-austro
Package name	ec.edu.uteq.bancoaustro
JDK	21
Java	21
Packaging	Jar

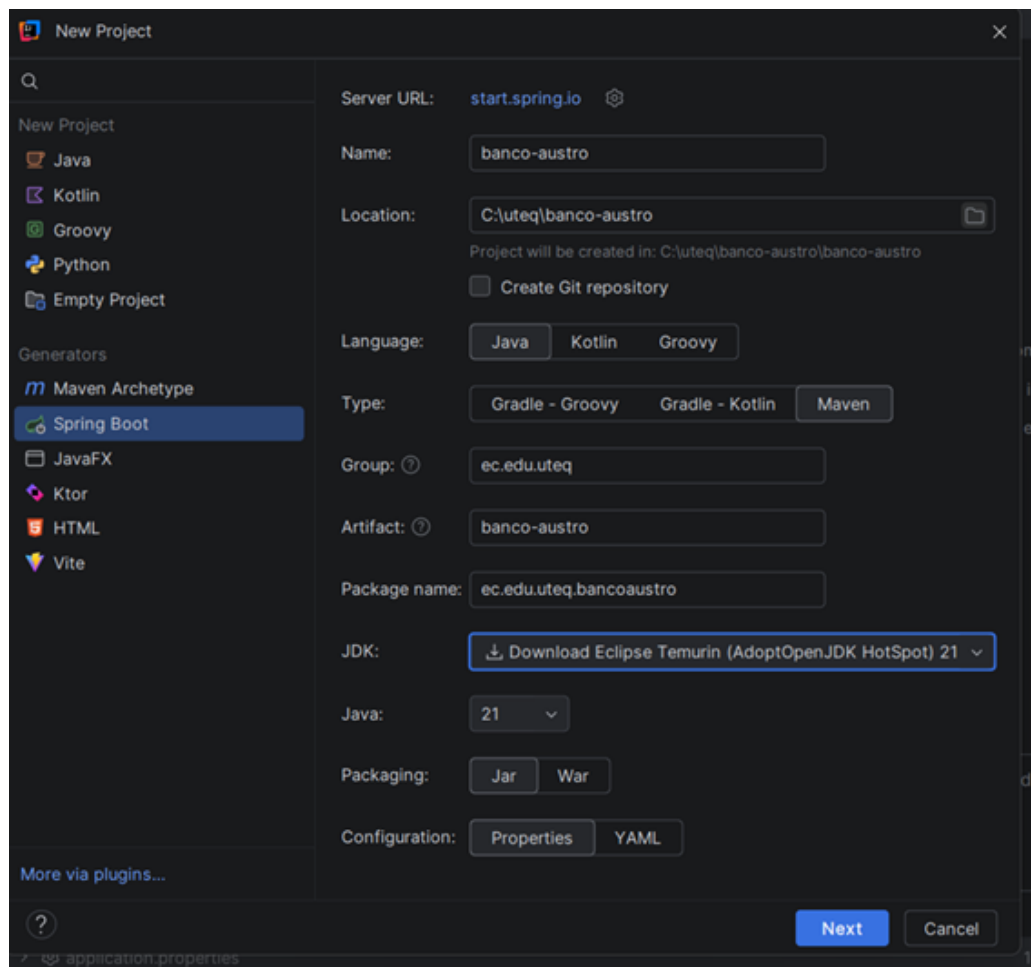


Figure 14: Configuración del proyecto

7.2 Elegir Spring Boot y dependencias

En la siguiente pantalla, seleccione la última versión estable de Spring Boot (3.3.x o 3.4.x) y marque las siguientes dependencias:

- **Spring Web** — para exponer endpoints REST.
- **JDBC API** — para conectarse a la base con JdbcTemplate.
- **PostgreSQL Driver** — para que Java hable con PostgreSQL.

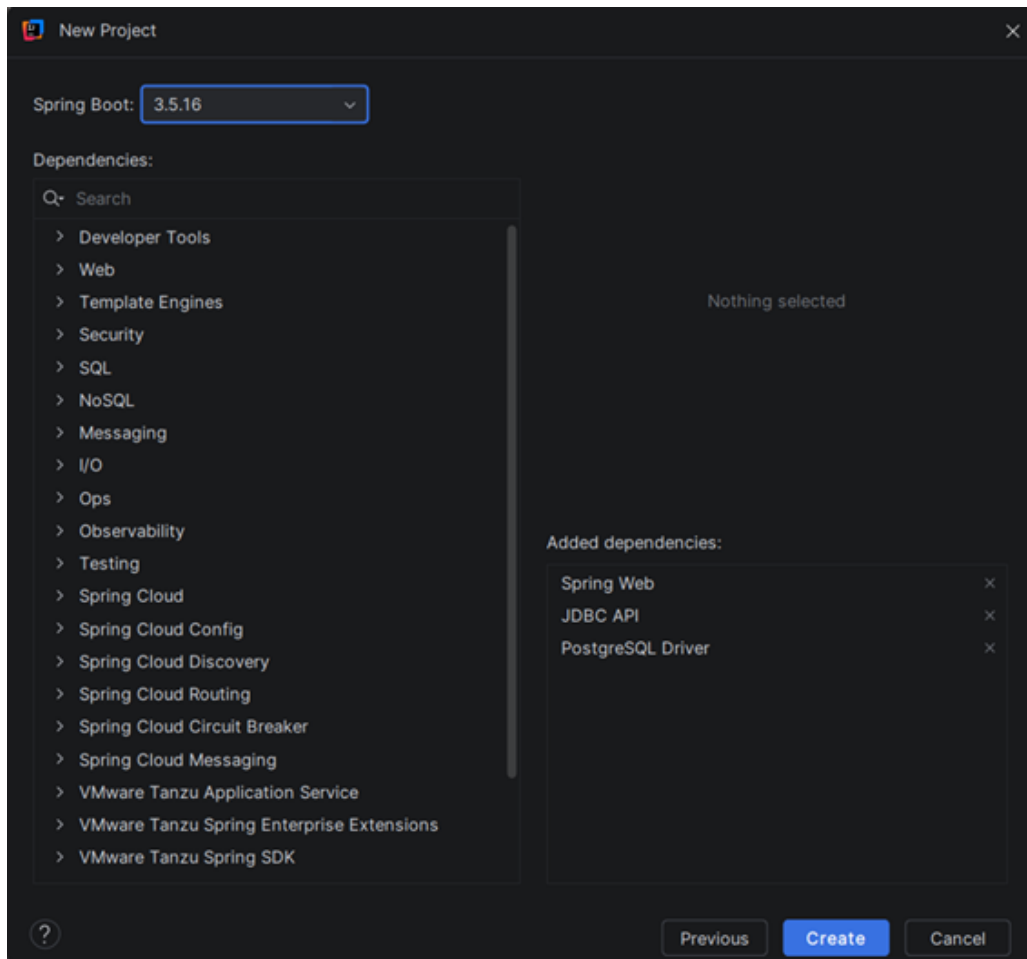


Figure 15: Dependencias seleccionadas

Pulse *Create*. IntelliJ descargará las dependencias de Maven y abrirá el proyecto. Espere a que el indicador de fondo indique que la indexación terminó.

7.3 Estructura del proyecto que se genera

7.4 Cambiar application.properties por application.yml

En el panel de proyecto, expanda `src/main/resources`. Clic derecho sobre `application.properties` → *Refactor* → *Rename* → póngale de nombre `application.yml`. Reemplace su contenido por:

```
1 server:
2   port: 8080
3
4 spring:
5   application:
6     name: banco-austro
7
```

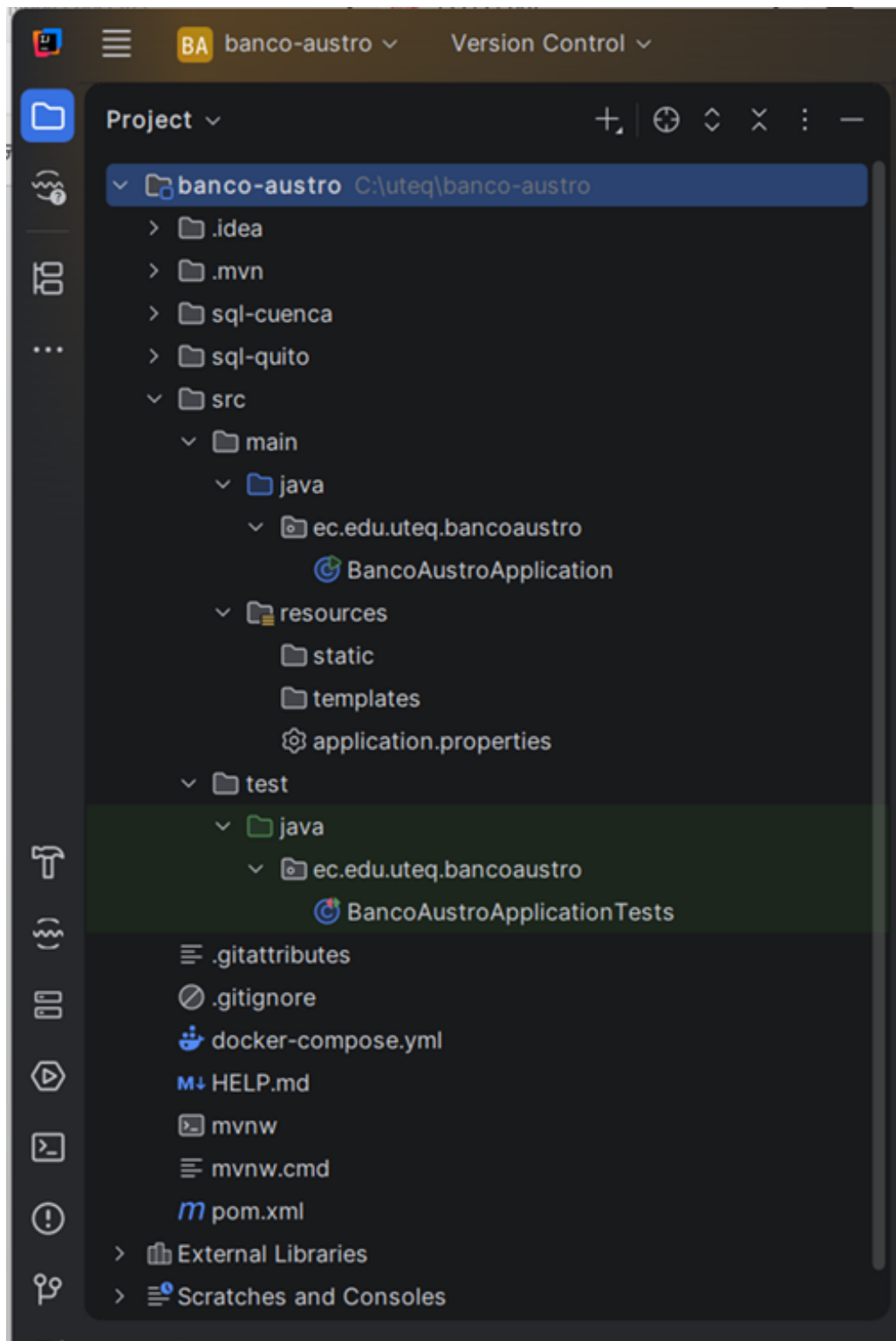


Figure 16: Estructura del proyecto

```

8 datasources:
9   cuenca:
10    url: jdbc:postgresql://localhost:5432/banco_cuenca
11    username: banco
12    password: banco123
13    driver-class-name: org.postgresql.Driver
14
15   quito:
16    url: jdbc:postgresql://localhost:5433/banco_quito
17    username: banco
18    password: banco123
19    driver-class-name: org.postgresql.Driver

```

Listing 14: application.yml

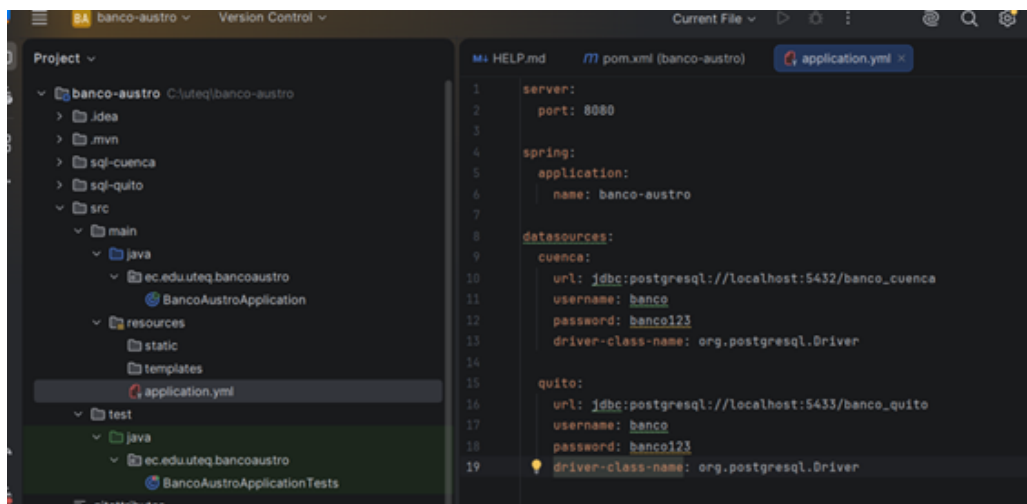


Figure 17: application.yml

8 Escribir el código Java de la aplicación

8.1 Organizar los paquetes

Se van a crear tres paquetes dentro de `ec.edu.uteq.bancoaustro`:

- **config**: contiene la clase que declara los dos `DataSource`.
- **service**: contiene la lógica de enrutamiento distribuido.
- **controller**: contiene los endpoints REST.

8.2 Clase de configuración de los dos DataSource

8.2.1 DataSourceConfig.java

Dentro del paquete `config`, clic derecho → *New* → *Java Class* → nombre `DataSourceConfig`. Reemplace el contenido por:

```

1 package ec.edu.uteq.bancoaustro.config;
2
3 import javax.sql.DataSource;
4
5 import org.springframework.beans.factory.annotation.Value;
6 import org.springframework.boot.jdbc.DataSourceBuilder;
7 import org.springframework.context.annotation.Bean;

```

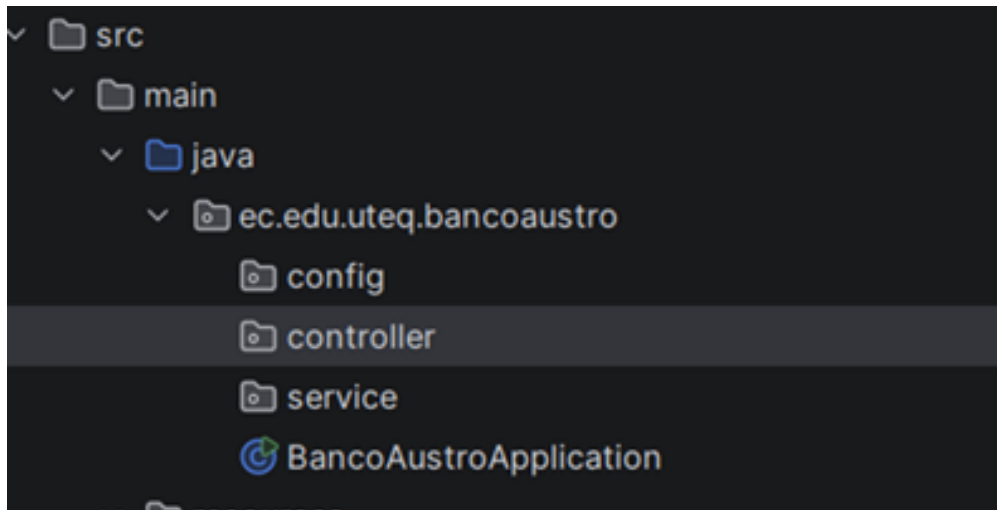



Figure 18: Organización de los paquetes

```
8 import org.springframework.context.annotation.Configuration;
9
10 @Configuration
11 public class DataSourceConfig {
12
13     @Value("${datasources.cuenca.url}")
14     private String cuencaUrl;
15
16     @Value("${datasources.cuenca.username}")
17     private String cuencaUser;
18
19     @Value("${datasources.cuenca.password}")
20     private String cuencaPass;
21
22     @Value("${datasources.quito.url}")
23     private String quitoUrl;
24
25     @Value("${datasources.quito.username}")
26     private String quitoUser;
27
28     @Value("${datasources.quito.password}")
29     private String quitoPass;
30
31     @Bean(name = "dsCuenca")
32     public DataSource dsCuenca() {
33         return DataSourceBuilder.create()
34             .url(cuencaUrl)
35             .username(cuencaUser)
36             .password(cuencaPass)
37             .driverClassName("org.postgresql.Driver")
38             .build();
39     }
40
41     @Bean(name = "dsQuito")
42     public DataSource dsQuito() {
43         return DataSourceBuilder.create()
44             .url(quitoUrl)
45             .username(quitoUser)
46             .password(quitoPass)
```

```

47         .driverClassName("org.postgresql.Driver")
48         .build();
49     }
50 }

```

Listing 15: DataSourceConfig.java

8.2.2 ConsultaDistribuidaService.java

En el paquete `service`, cree la clase `ConsultaDistribuidaService` con este contenido:

```

1  package ec.edu.uteq.bancoaustro.service;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.Map;
6  import javax.sql.DataSource;
7
8  import org.springframework.beans.factory.annotation.Qualifier;
9  import org.springframework.jdbc.core.JdbcTemplate;
10 import org.springframework.stereotype.Service;
11
12 @Service
13 public class ConsultaDistribuidaService {
14
15     private final JdbcTemplate jdbcCuenca;
16     private final JdbcTemplate jdbcQuito;
17
18     public ConsultaDistribuidaService(
19         @Qualifier("dsCuenca") DataSource dsCuenca,
20         @Qualifier("dsQuito") DataSource dsQuito
21     ) {
22         this.jdbcCuenca = new JdbcTemplate(dsCuenca);
23         this.jdbcQuito = new JdbcTemplate(dsQuito);
24     }
25
26     public Map<String, Object> consultarSaldo(String numero) {
27         JdbcTemplate destino = enrutar(numero);
28
29         String sql = "SELECT numero, saldo, oficina FROM cuentas
30             WHERE numero = ?";
31
32         List<Map<String, Object>> filas = destino.queryForList(sql,
33             numero);
34
35         if (filas.isEmpty()) {
36             return Map.of(
37                 "error", "Cuenta no encontrada",
38                 "numero", numero
39             );
40         }
41
42         return filas.get(0);
43     }
44
45     public List<Map<String, Object>> listarTodosLosClientes() {
46         String sql = "SELECT cedula, nombre, ciudad FROM clientes";
47     }
48 }

```

```

46         List<Map<String, Object>> union = new ArrayList<>();
47
48         union.addAll(jdbcCuenca.queryForList(sql));
49         union.addAll(jdbcQuito.queryForList(sql));
50
51         return union;
52     }
53
54     private JdbcTemplate enrutar(String numero) {
55         if (numero == null || numero.length() < 2) {
56             throw new IllegalArgumentException(
57                 "Numero de cuenta invalido: " + numero
58             );
59         }
60
61         String prefijo = numero.substring(0, 2);
62
63         return switch (prefijo) {
64             case "22" -> jdbcCuenca;
65             case "17" -> jdbcQuito;
66             default -> throw new IllegalArgumentException(
67                 "Prefijo de oficina no reconocido: " + prefijo
68             );
69         };
70     }
71 }

```

Listing 16: ConsultaDistribuidaService.java

8.2.3 BancoController.java

En el paquete controller, cree la clase BancoController con este contenido:

```

1  package ec.edu.uteq.bancoaustro.controller;
2
3  import java.util.List;
4  import java.util.Map;
5
6  import ec.edu.uteq.bancoaustro.service.ConsultaDistribuidaService;
7  import org.springframework.web.bind.annotation.GetMapping;
8  import org.springframework.web.bind.annotation.PathVariable;
9  import org.springframework.web.bind.annotation.RequestMapping;
10 import org.springframework.web.bind.annotation.RestController;
11
12 @RestController
13 @RequestMapping("/api/banco")
14 public class BancoController {
15
16     private final ConsultaDistribuidaService service;
17
18     public BancoController(ConsultaDistribuidaService service) {
19         this.service = service;
20     }
21
22     @GetMapping("/saldo/{numero}")
23     public Map<String, Object> saldo(@PathVariable String numero) {
24         return service.consultarSaldo(numero);
25     }

```

```
26
27     @GetMapping("/clientes")
28     public List<Map<String, Object>> clientes() {
29         return service.listarTodosLosClientes();
30     }
31 }
```

Listing 17: BancoController.java

8.3 Estructura del proyecto Java en este punto

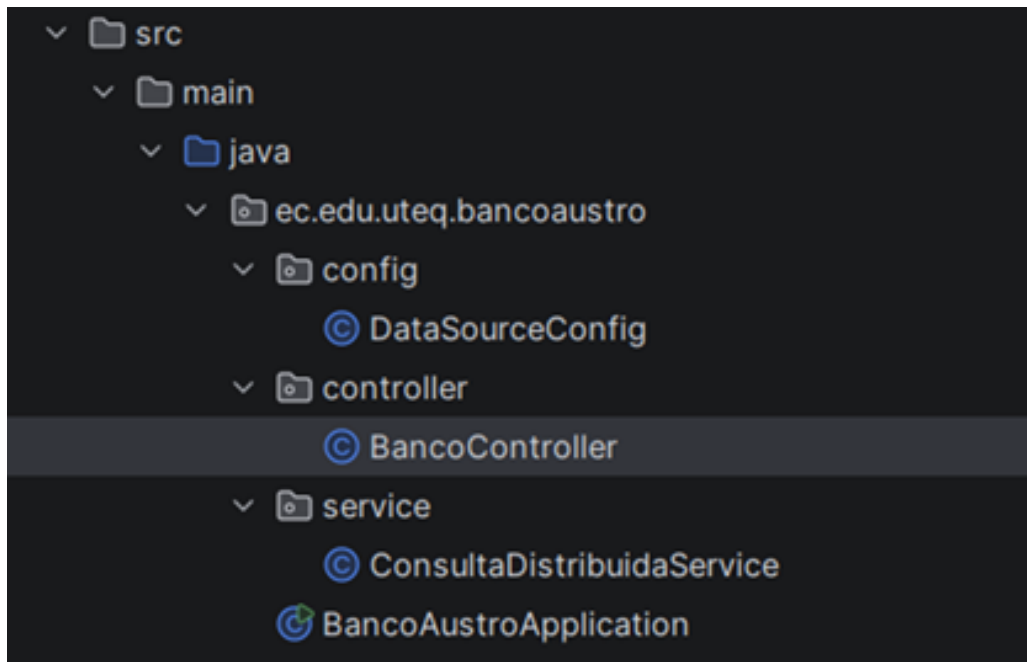


Figure 19: Estructura del proyecto Java actualizado

9 Ejecutar y probar la aplicación

9.1 Levantar la aplicación Spring Boot

En el panel del proyecto, abra la clase `BancoAustroApplication.java`. Haga clic en el triángulo verde que aparece a la izquierda del nombre de la clase, o al lado del método `main`, y elija *Run 'BancoAustroApplication'*. También funciona el atajo `Shift+F10`. En la parte inferior aparecerá la consola de ejecución con los logs de Spring Boot. Espere a las líneas:

```
1 Tomcat started on port(s): 8080 (http)
2 Started BancoAustroApplication in X.XXX seconds
```

Listing 18: Confirmación de arranque de la aplicación

Mientras esa línea no aparezca, la aplicación todavía no está lista para recibir peticiones.

9.2 Probar los endpoints con Postman

Una vez ejecutada la aplicación Spring Boot, se realizaron pruebas mediante peticiones `GET` en Postman para verificar que el sistema consulte correctamente los nodos distribuidos de Cuenca y Quito.

9.2.1 Primera petición: consultar saldo en el nodo Cuenca

Se abrió Postman y se seleccionó el método GET. Luego se ingresó la siguiente URL:

```
1 http://localhost:8080/api/banco/saldo/2201000001
```

Listing 19: Endpoint de saldo — nodo Cuenca

La respuesta esperada es un JSON similar al siguiente:

```
1 {
2   "numero": "2201000001",
3   "saldo": 15000.00,
4   "oficina": "CUENCA"
5 }
```

Listing 20: Respuesta esperada — nodo Cuenca

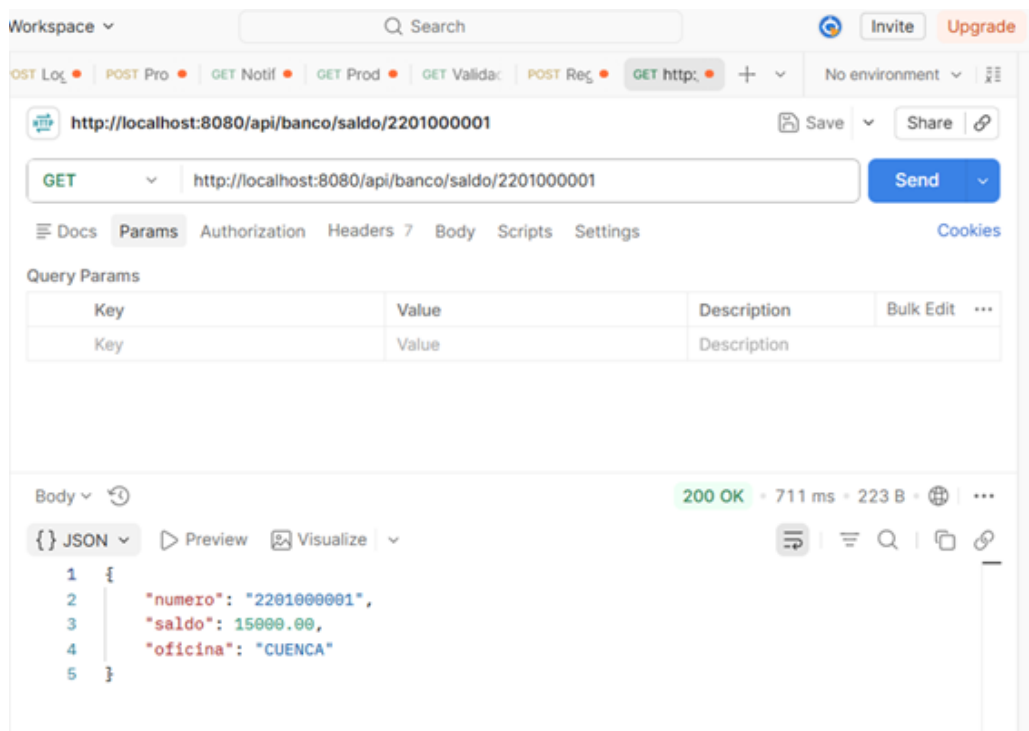


Figure 20: Prueba 1 — consulta de saldo en el nodo Cuenca

Esta prueba demuestra que la aplicación identificó el prefijo 22 del número de cuenta y dirigió la consulta al nodo correspondiente de Cuenca.

9.2.2 Segunda petición: consultar saldo en el nodo Quito

Luego se cambió la URL por la siguiente:

```
1 http://localhost:8080/api/banco/saldo/1701000001
```

Listing 21: Endpoint de saldo — nodo Quito

La respuesta esperada es:

```
1 {
2   "numero": "1701000001",
3   "saldo": 8000.00,
4   "oficina": "QUITO"
```

5 }

Listing 22: Respuesta esperada — nodo Quito

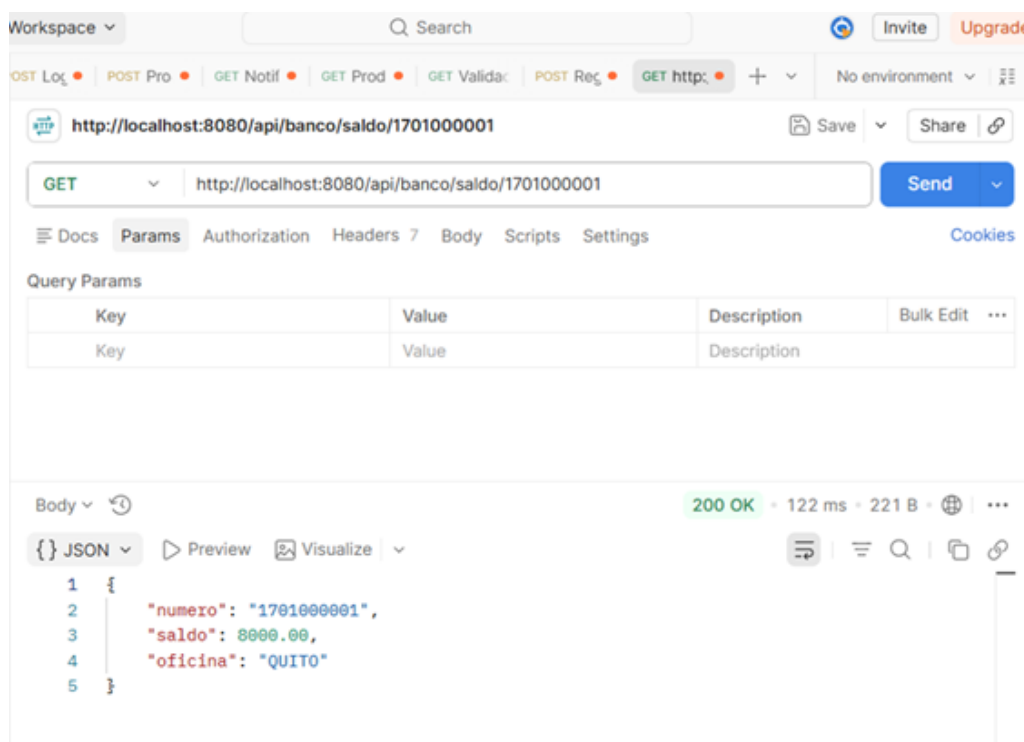


Figure 21: Prueba 2 — consulta de saldo en el nodo Quito

Esta prueba demuestra que la aplicación identificó el prefijo 17 y dirigió la consulta al nodo correspondiente de Quito.

9.2.3 Tercera petición: unión de clientes de ambos nodos

Finalmente, se probó la consulta que une los datos de los dos nodos mediante la siguiente URL:

```
1 http://localhost:8080/api/banco/clientes
```

Listing 23: Endpoint de unión de clientes

La respuesta debe mostrar un arreglo JSON con seis registros: los tres clientes del nodo Cuenca y los tres clientes del nodo Quito.

Esta prueba evidencia la operación de unión sobre los fragmentos distribuidos, permitiendo consultar información de ambos nodos desde una sola aplicación.

10 Prueba de tolerancia a fallos

Uno de los cinco requisitos del caso era: *si una oficina se cae, las demás siguen operando*. Se comprueba empíricamente.

10.1 Detener el nodo Quito

Con la aplicación Spring Boot todavía corriendo, abra un nuevo `cmd.exe` y ejecute:

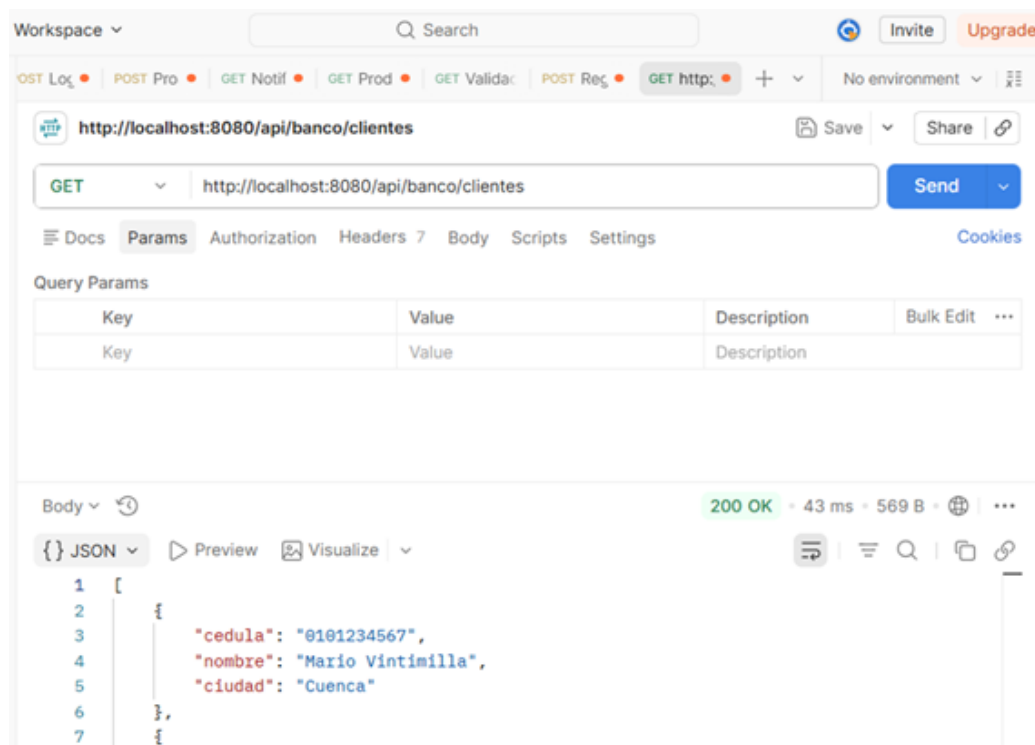


Figure 22: Prueba 3 — unión de clientes de ambos nodos

```
1 docker stop banco-quito
2 docker compose ps
```

Listing 24: Detener el contenedor del nodo Quito

El listado debe mostrar a **banco-cuenca** como *running* y a **banco-quito** como *exited*. En Docker Desktop, el punto verde de Quito pasará a gris.

10.2 Repetir las pruebas con un nodo caído

Regrese a Postman y vuelva a lanzar las peticiones:

- GET `/api/banco/saldo/2201000001` → sigue respondiendo correctamente porque la cuenta está en Cuenca, que sigue viva.
- GET `/api/banco/saldo/1701000001` → falla con un error de conexión al puerto 5433. La aplicación no sabe cómo llegar a Quito.
- GET `/api/banco/clientes` → falla porque el método actual intenta consultar los dos nodos y no captura el error del nodo caído.

Este resultado deja en evidencia que, si bien la fragmentación horizontal permite aislar el impacto de una caída sobre las consultas dirigidas a un único nodo, la operación de *unión* necesita manejo explícito de excepciones para degradar el servicio de forma controlada en lugar de fallar por completo.

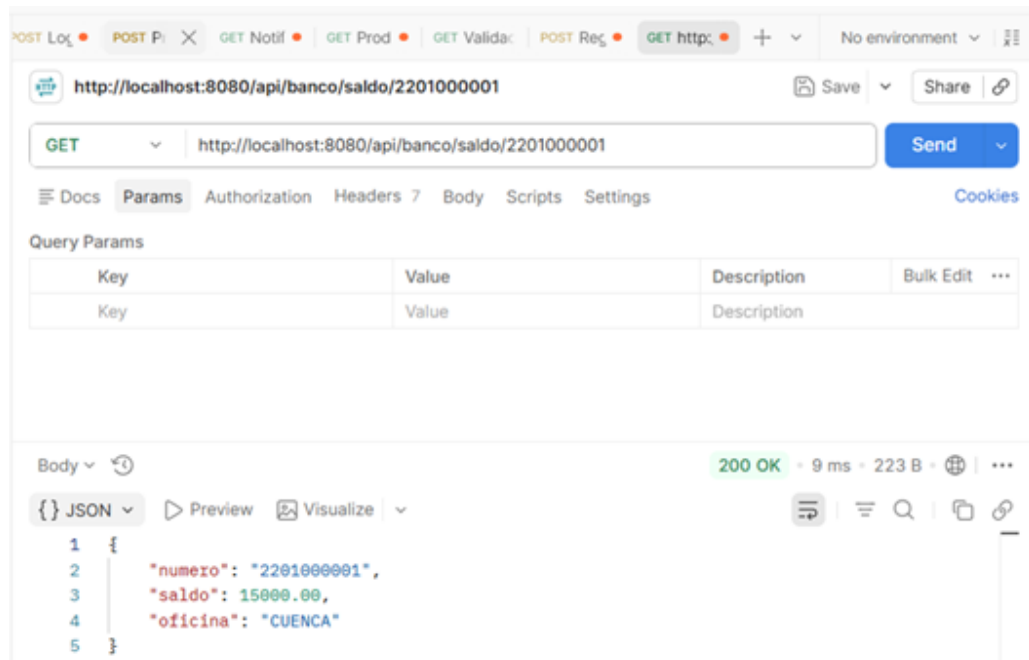


Figure 23: Tolerancia a fallos — la consulta a Cuenca sigue funcionando

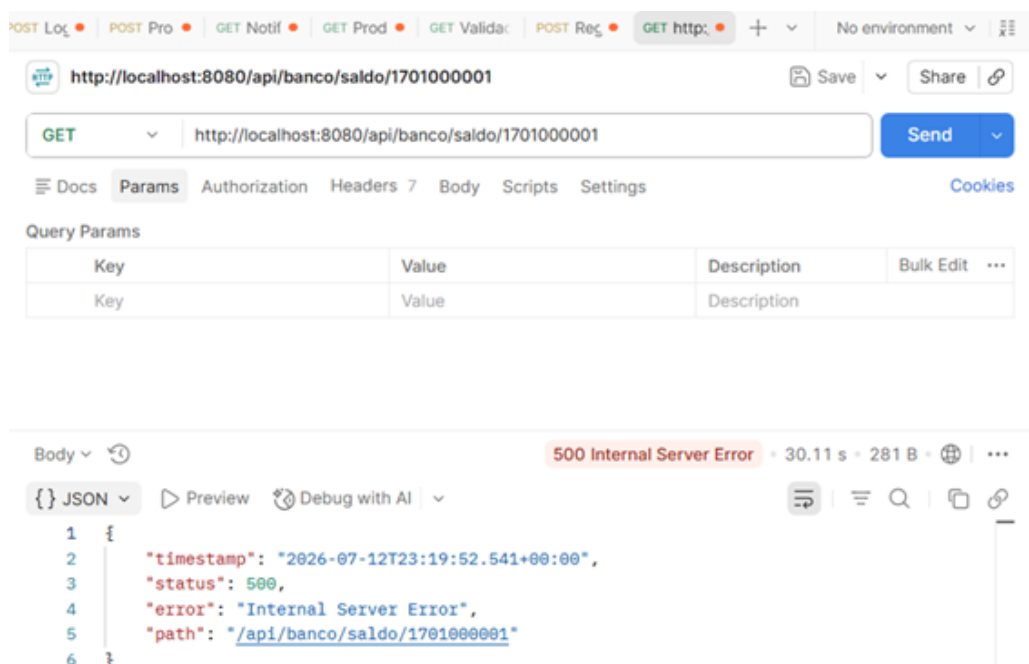


Figure 24: Tolerancia a fallos — la consulta a Quito falla

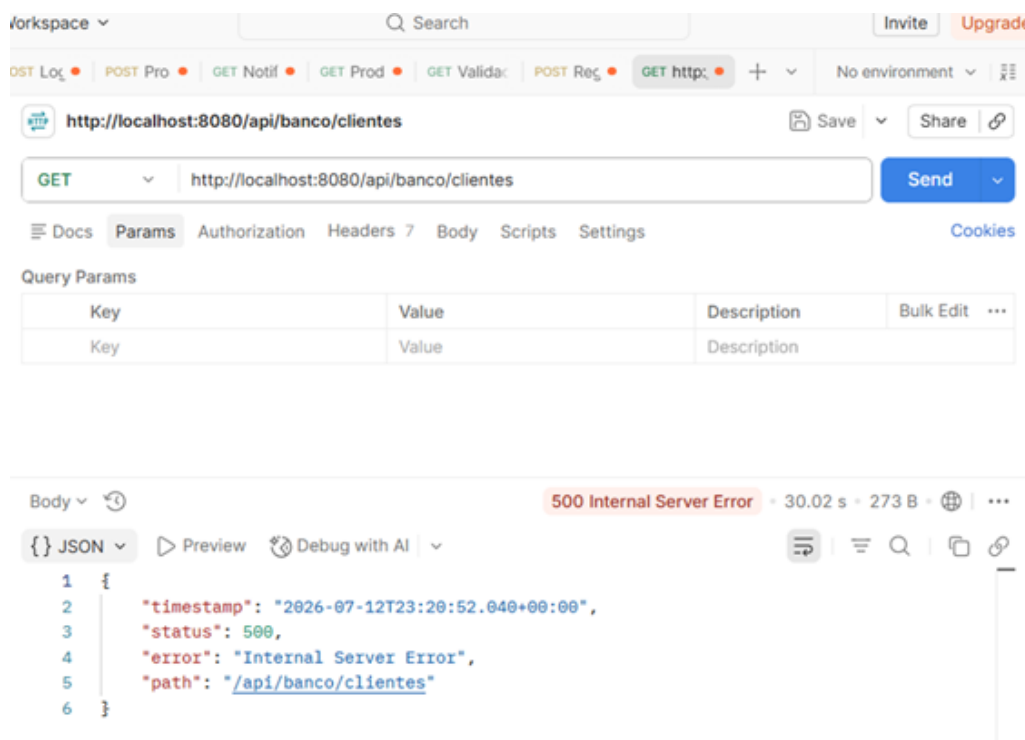


Figure 25: Tolerancia a fallos — la unión de clientes falla sin manejo de excepciones

11 Conclusiones

- La fragmentación horizontal permitió distribuir físicamente los datos del banco en dos nodos PostgreSQL independientes, cada uno responsable únicamente de su oficina.
- El enrutamiento por prefijo de cuenta, implementado en `ConsultaDistribuidaService`, demostró dirigir correctamente cada consulta al nodo correspondiente.
- La operación de unión sobre ambos nodos funciona correctamente en condiciones normales, pero requiere manejo de excepciones para tolerar la caída parcial del sistema.
- Las pruebas de tolerancia a fallos confirmaron que las consultas dirigidas a un nodo activo no se ven afectadas por la caída de otro nodo, cumpliendo parcialmente con el requisito de disponibilidad del caso de estudio.